

Tarea_04

September 26, 2025

1 Examen Modulo 1 - Iranian Churn

- Centeno Amado Eduardo
- Orozpe Frias Erick Adrian
- Pineda Gaona Maricruz
- Rodríguez Teodores Armando Jair
- Velasco Martell Dayra

2 Instrucciones

Pasos a seguir

1. Descarga el dataset de UCI en el siguiente link:
<https://archive.ics.uci.edu/dataset/563/iranian+churn+dataset>

Explora el sitio para tener información de las variables y sus consideraciones.
2. Verifica si hay valores ausentes. Si es así, considera imputarlos o eliminar los registros.
3. Elabora el análisis exploratorio de los datos:
 - a) Describe la distribución de la variable objetivo (Churn). ¿Está balanceada?
 - b) Calcula estadísticas descriptivas para variables numéricas. ¿Qué puedes inferir sobre ellas? ¿Hay valores atípicos?
 - c) Analiza las variables categóricas. Considera normalizarlas si lo crees necesario.
 - d) Remueve valores atípicos si lo consideras pertinente. Justifica el porqué.
 - e) Explora correlaciones entre las variables numéricas. ¿Tienen sentido estas correlaciones? ¿Existen problemas de multicolinealidad?
 - f) Identifica variables que parecen influir más en Churn.
4. Codifica las variables categóricas, dependiendo si son ordinales o nominales.
5. Escala las variables numéricas. Utiliza el escalador que creas conveniente.

6. Aplica mutual information, WoE e IV para conocer el poder predictivo que tendrían las variables explicativas con la variable Churn. ¿Qué variables parecen explicarla bien?
7. ¿Consideras necesario realizar PCA para reducir dimensiones? Si es así, hazlo. Considera el número de componentes necesario.

3 Bibliotecas

```
[1]: # Instalar la primera vez de ejecución
    # pip install pyod
```

```
[2]: #General
import pandas as pd
import numpy as np
import re #RegEx

#Graficas
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots

#Valores atipicos
from pyod.models.knn import KNN
from pyod.models.iforest import IForest
from pyod.models.mcd import MCD

#Reduccion de dimension
from sklearn.decomposition import PCA
from sklearn.manifold import MDS
from sklearn.manifold import TSNE

#Escaladores
from sklearn.preprocessing import MinMaxScaler, RobustScaler, StandardScaler

#Clustering
from sklearn.cluster import KMeans, DBSCAN
from sklearn.metrics import silhouette_score
from sklearn.mixture import GaussianMixture
from sklearn.neighbors import NearestNeighbors
from sklearn.metrics import calinski_harabasz_score, davies_bouldin_score

#Pruebas estadisticas
from scipy.stats import kruskal, chi2_contingency
```

```
#Codificadores
from sklearn.preprocessing import LabelEncoder, OrdinalEncoder, OneHotEncoder

# Relevancia de las variables
from sklearn.feature_selection import mutual_info_classif
```

4 Extracción de Datos

Descarga el dataset de UCI en el siguiente link: <https://archive.ics.uci.edu/dataset/563/iranian+churn+dataset>

Explora el sitio para tener información de las variables y sus consideraciones.

```
[3]: df = pd.read_csv(r"https://raw.githubusercontent.com/armandoteo/Datos-CDD/refs/heads/main/Customer%20Churn.csv")
```

```
[4]: print("Dimensión", df.shape)
```

Dimensión (3150, 14)

Este conjunto de datos fue recolectado aleatoriamente de la base de datos de una compañía de telecomunicaciones iraní durante un periodo de 12 meses. Consta de un total de 3,150 registros, cada uno representando a un cliente, con información en 13 columnas.

Variable Name	Role	Type	Description
Call Failure	Feature	Integer	Número de llamadas fallidas
Complains	Feature	Binary	Quejas del cliente (0 = No, 1 = Sí)
Subscription Length	Feature	Integer	Meses totales de suscripción
Charge Amount	Feature	Integer	Monto de cargos (0 = más bajo, 9 = más alto)
Seconds of Use	Feature	Integer	Total de segundos en llamadas
Frequency of use	Feature	Integer	Número total de llamadas realizadas
Frequency of SMS	Feature	Integer	Número total de mensajes SMS enviados
Distinct Called Numbers	Feature	Integer	Cantidad de números de teléfono distintos llamados
Age Group	Feature	Integer	Grupo de edad (1 = más joven, 5 = mayor)
Tariff Plan	Feature	Integer	Plan de tarifa (1 = prepago, 2 = contrato)
Status	Feature	Binary	Estado del cliente (1 = activo, 2 = inactivo)
Age	Feature	Integer	Edad exacta del cliente
Customer Value	Feature	Continuous	Valor calculado del cliente
Churn	Target	Binary	Variable objetivo: 1 = Churn (abandono), 0 = No Churn (retención)

Los atributos incluidos en este conjunto de datos son: fallas en llamadas, frecuencia de SMS, número de quejas, número de llamadas distintas, duración de la suscripción, grupo de edad, monto de cargos, tipo de servicio, segundos de uso, estado, frecuencia de uso y valor del cliente.

Todos los atributos, excepto la variable *churn*, corresponden a datos agregados de los primeros 9 meses. Las etiquetas de *churn* reflejan el estado de los clientes al finalizar los 12 meses. Los tres meses restantes representan el periodo definido como brecha de planeación. Data Set Recolectado de: <https://archive.ics.uci.edu/dataset/563/iranian+churn+dataset>

5 Valores Ausentes

```
[5]: def is_na_df(df):
    #Establecemos el resumen de NAs por columna
    na_summary = (
        df.isna().sum()
        .reset_index()
        .rename(columns={"index": "columna", 0: "n_missing"})
    )

    #Tomamos totales y prporciones
    na_summary["total"] = len(df)
    na_summary["prop_missing"] = na_summary["n_missing"] / na_summary["total"]
    na_summary["prop_original"] = 1 - na_summary["prop_missing"]

    #Filtramos solo columnas con al menos 1 NA
    na_summary = na_summary[na_summary["n_missing"] > 0].reset_index(drop=True)

    #Agregamos fila resumen global
    total_missing = df.isna().sum().sum()
    total_elements = df.size
    prop_missing_global = total_missing / total_elements
    prop_original_global = 1 - prop_missing_global

    resumen = pd.DataFrame({
        "columna": ["TOTAL"],
        "n_missing": [total_missing],
        "total": [total_elements],
        "prop_missing": [prop_missing_global],
        "prop_original": [prop_original_global]
    })

    na_summary = pd.concat([na_summary, resumen], ignore_index=True)

    return na_summary
```

```
[6]: is_na_df(df)
```

```
[6]:  columna  n_missing  total  prop_missing  prop_original
0    TOTAL           0   44100           0.0           1.0
```

La base no trae valores ausentes como se comenta en el sitio web de donde se extrajo el dataset.

6 Análisis Exploratorio

Describe la distribución de la variable objetivo (Churn). ¿Está balanceada?

```
[7]: conteo = df['Churn'].value_counts()
display(pd.concat([df['Churn'].value_counts(), df['Churn'].
    ↪value_counts(normalize=True).apply(lambda x: '{:.2%}'.format(x))], axis=1))

balance_ratio = conteo.min() / conteo.max()

print(f"\nÍndice de balance de {balance_ratio:.2%}", end=" por lo que ")

if balance_ratio > 0.7:
    print("la variable está relativamente balanceada.")
elif balance_ratio > 0.3:
    print("hay cierto desbalance, podría afectar algunos modelos.")
else:
    print("hay fuerte desbalance.")
```

	count	proportion
Churn		
0	2655	84.29%
1	495	15.71%

Índice de balance de 18.64% por lo que hay fuerte desbalance.

La distribución de la variable objetivo refleja que el 84.29% de los clientes permanecieron con la compañía, mientras que el 15.71% abandonaron el servicio. Esto evidencia un desbalance en la variable de churn, dado que la mayoría de los registros corresponden a clientes retenidos frente a una menor proporción de clientes en fuga.

Calcula estadísticas descriptivas para variables numéricas. ¿Qué puedes inferir sobre ellas? ¿Hay valores a

```
[8]: #Definimos un objeto para graficar
class analyze:
    # Inicializamos la clase
    def __init__(self, df, num_vars, cat_vars):
        self.df = df
        self.num_vars = num_vars
        self.cat_vars = cat_vars

    #Método para graficar variables numéricas
    def plot_num(self):
        for var in self.num_vars:
            fig = make_subplots(rows=1, cols=2, subplot_titles=(f"{var} - 
    ↪Boxplot", f"{var} - Histograma"))

            # Boxplot
```

```

fig.add_trace(
    go.Box(y=self.df[var], name="Boxplot", marker_color="#008080"),
    row=1, col=1
)

# Histograma
fig.add_trace(
    go.Histogram(x=self.df[var], name="Histograma",
        marker_color="#00a77d", nbinsx=50,
        histnorm='probability density'),
    row=1, col=2
)

fig.update_layout(title_text=f"Variable: {var}", height=400,
↪width=800)
fig.show()

#Método para graficar variables categóricas
def plot_cat(self):
    for var in self.cat_vars:
        value_counts = self.df[var].value_counts()
        labels = value_counts.index.astype(str)
        values = value_counts.values

        fig = go.Figure(
            data=[go.Pie(labels=labels, values=values,
↪textinfo='percent+label')]
        )

        fig.update_layout(title_text=f"Variable: {var}", height=400,
↪width=500)
        fig.show()

#Método para graficar la matriz de correlación de numéricas
def corr_matrix(self, color, values: bool = False):
    corr_matrix = self.df[self.num_vars].corr()

    if values:
        fig = go.Figure(data=go.Heatmap(
            z=corr_matrix.values,
            x=corr_matrix.columns,
            y=corr_matrix.columns,
            colorscale=color,
            text=corr_matrix.round(1).values, # valores de correlación
↪con 1 decimal
            texttemplate="%{text}", # mostrar texto en cada
↪celda

```

```

        textfont={"size":10, "color":"black"} # estilo de fuente
    ))

else:
    fig = go.Figure(data=go.Heatmap(
        z=corr_matrix.values,
        x=corr_matrix.columns,
        y=corr_matrix.columns,
        colorscale=color
    ))

    fig.update_layout(
        title='Matriz de Correlación',
        width=800,
        height=600
    )

fig.show()

```

```

[9]: # Definimos las variables categoricas
var_cat = ['Complains', 'Charge Amount', 'Age Group', 'Tariff_
↳ Plan', 'Status', 'Churn']

# Definimos las variables numericas
var_num = list(set(df.columns) - set(var_cat))

```

```

[10]: analisis = analyze(df, var_num, var_cat)

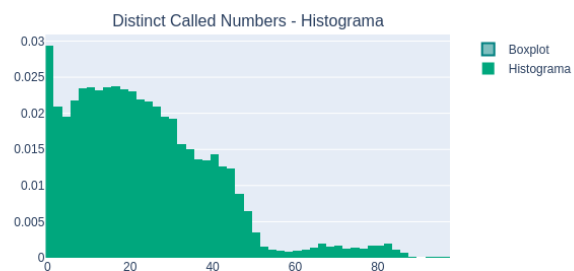
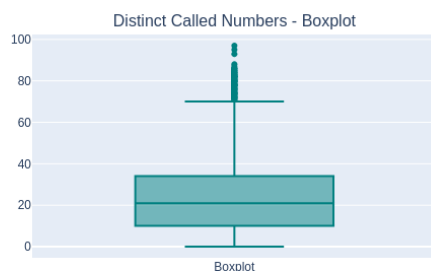
```

```

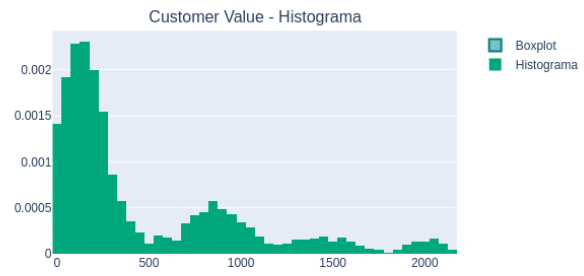
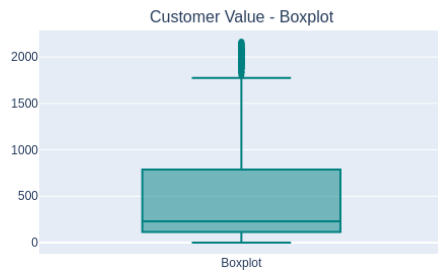
[11]: analisis.plot_num()

```

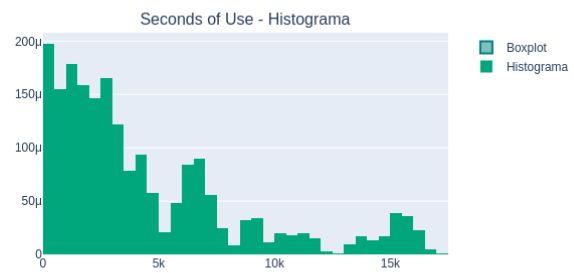
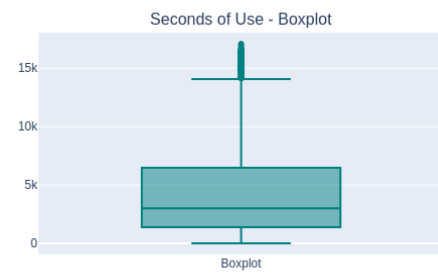
Variable: Distinct Called Numbers



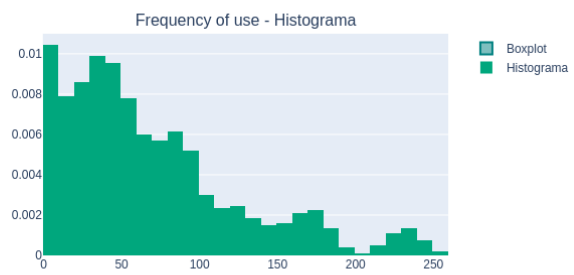
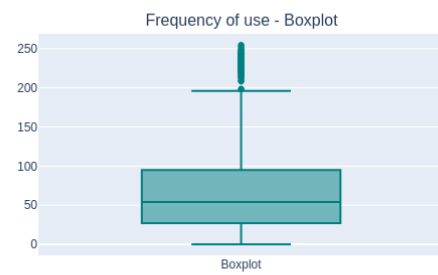
Variable: Customer Value



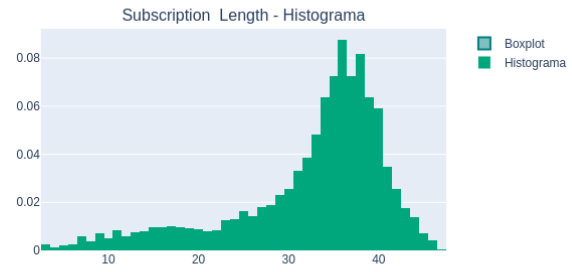
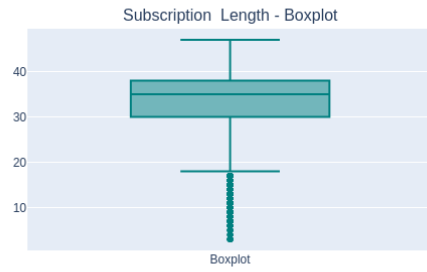
Variable: Seconds of Use



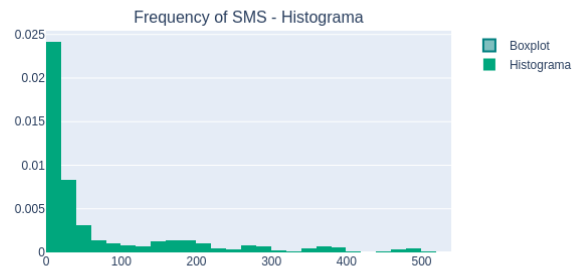
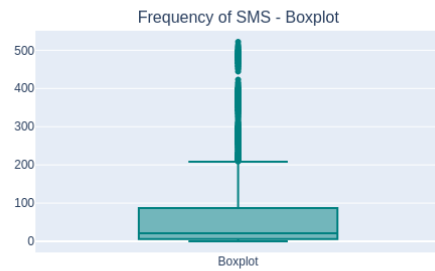
Variable: Frequency of use



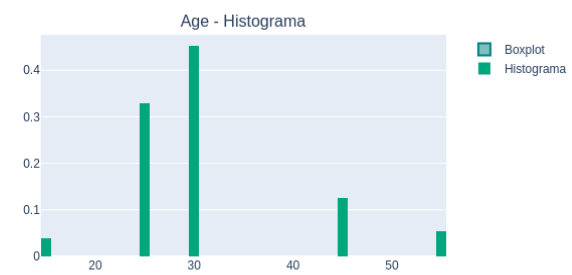
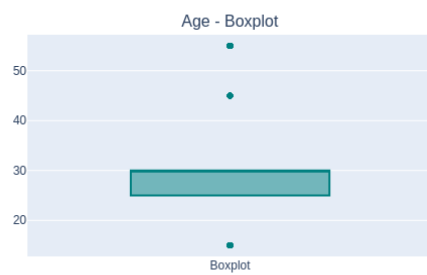
Variable: Subscription Length



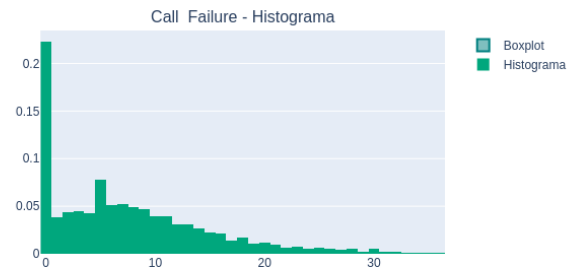
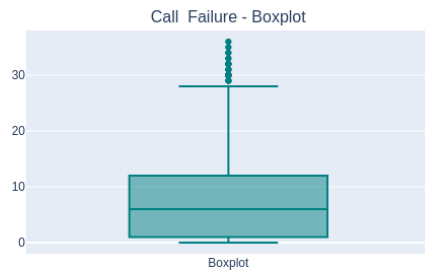
Variable: Frequency of SMS



Variable: Age



Variable: Call Failure



```
[12]: perc=[0.01,0.05,0.25,0.5,0.75,0.95,0.99]
```

```
[13]: # Vemos las estadísticas
df[var_num].describe(percentiles=perc).T
```

```
[13]:
```

	count	mean	std	min	1%	5%	\
Distinct Called Numbers	3150.0	23.509841	17.217337	0.0	0.0	1.000	
Customer Value	3150.0	470.972916	517.015433	0.0	0.0	10.335	
Seconds of Use	3150.0	4472.459683	4197.908687	0.0	0.0	54.500	
Frequency of use	3150.0	69.460635	57.413308	0.0	0.0	1.000	
Subscription Length	3150.0	32.541905	8.573482	3.0	7.0	13.000	
Frequency of SMS	3150.0	73.174921	112.237560	0.0	0.0	0.000	
Age	3150.0	30.998413	8.831095	15.0	15.0	25.000	
Call Failure	3150.0	7.627937	7.263886	0.0	0.0	0.000	

	25%	50%	75%	95%	\
Distinct Called Numbers	10.00000	21.00	34.00000	51.00	
Customer Value	113.80125	228.48	788.38875	1587.68	
Seconds of Use	1391.25000	2990.00	6478.25000	15020.50	
Frequency of use	27.00000	54.00	95.00000	184.55	
Subscription Length	30.00000	35.00	38.00000	42.00	
Frequency of SMS	6.00000	21.00	87.00000	356.55	
Age	25.00000	30.00	30.00000	55.00	
Call Failure	1.00000	6.00	12.00000	22.00	

	99%	max
Distinct Called Numbers	80.0000	97.00
Customer Value	2059.9036	2165.28
Seconds of Use	16127.6500	17090.00
Frequency of use	238.5100	255.00
Subscription Length	45.0000	47.00
Frequency of SMS	483.0000	522.00
Age	55.0000	55.00

Call	Failure	30.0000	36.00
------	---------	---------	-------

```
[14]: # Intervalo de edades por Age Group
df.groupby('Age Group')['Age'].agg(['count', 'min', 'mean', 'median', 'max'])
```

```
[14]:
```

	count	min	mean	median	max
Age Group					
1	123	15	15.0	15.0	15
2	1037	25	25.0	25.0	25
3	1425	30	30.0	30.0	30
4	395	45	45.0	45.0	45
5	170	55	55.0	55.0	55

```
[15]: # Clasificación de la variable objetivo por Tipo de plan
aux = df.loc[:, ['Churn', 'Tariff Plan', 'Age']]
aux['Churn_recat'] = aux['Churn'].apply(lambda x: 'Abandono' if x == 1 else
↳ 'Retención')
aux['Tariff Plan_recat'] = aux['Tariff Plan'].apply(lambda x: 'Prepago' if x ==
↳ 1 else 'Contractual')
aux.groupby(['Churn_recat', 'Tariff Plan_recat']).agg('count').iloc[:,0].
↳ reset_index(name='Count')
```

```
[15]:
```

	Churn_recat	Tariff Plan_recat	Count
0	Abandono	Contractual	6
1	Abandono	Prepago	489
2	Retención	Contractual	239
3	Retención	Prepago	2416

Puede no aportarnos para predecir si el cliente abandona por la alta concentración de prepago en los planes de tarifa.

```
[16]: # Clasificación de la variable objetivo por Estatus del Cliente
aux = df.loc[:, ['Churn', 'Status', 'Age']]
aux['Churn_recat'] = aux['Churn'].apply(lambda x: 'Abandono' if x == 1 else
↳ 'Retención')
aux['Status_recat'] = aux['Status'].apply(lambda x: 'Activo' if x == 1 else
↳ 'Inactivo')
aux.groupby(['Churn_recat', 'Status_recat']).agg('count').iloc[:,0].
↳ reset_index(name='Count')
```

```
[16]:
```

	Churn_recat	Status_recat	Count
0	Abandono	Activo	125
1	Abandono	Inactivo	370
2	Retención	Activo	2243
3	Retención	Inactivo	412

Puede aportarnos para predecir si el cliente abandona por que tenemos distribución de clientes activos e inactivos, además vemos cierta relación de abandono por clientes inactivos lo cual hace

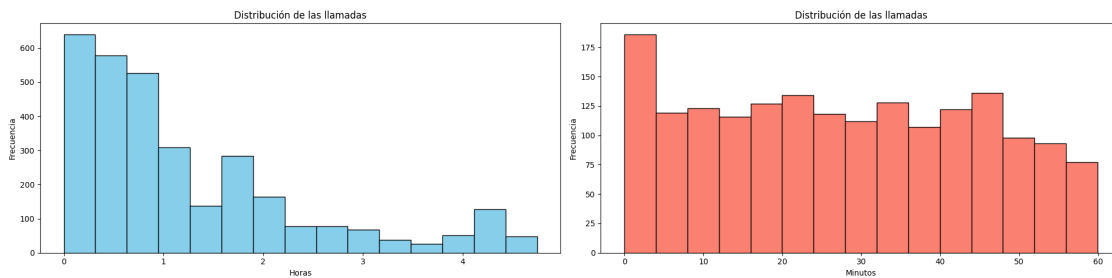
sentido.

```
[17]: # Crear figura con dos ejes (1 fila, 2 columnas)
fig, axes = plt.subplots(1, 2, figsize=(20, 5))

# Histograma en el primer eje
axes[0].hist(df['Seconds of Use']/3600, bins=15, color="skyblue",
             edgecolor="black")
axes[0].set_title("Distribución de las llamadas")
axes[0].set_xlabel("Horas")
axes[0].set_ylabel("Frecuencia")

# Histograma en el segundo eje
axes[1].hist(df.loc[:, ['Seconds of Use']].apply(lambda x: x/60).query("`Seconds of Use` <= 60"), bins=15, color="salmon", edgecolor="black")
axes[1].set_title("Distribución de las llamadas")
axes[1].set_xlabel("Minutos")
axes[1].set_ylabel("Frecuencia")

plt.tight_layout()
plt.show()
```



```
[18]: # H0: El número de llamadas realizadas es igual a la cantidad de números
       distintos marcados por el cliente

aux2 = (df['Frequency of use'] == df['Distinct Called Numbers'])

display(
    (
        pd.concat(
            [aux2.value_counts(), aux2.value_counts(normalize=True).
             apply(lambda x: '{:.2%}'.format(x))],
            axis=1
        )
    )
)
```

	count	proportion
False	2976	94.48%
True	174	5.52%

Call Failure

El rango intercuantil está entre 1 y 12 llamadas con falla, pero algunos casos extremos elevan el promedio.

Subscription Length

El rango intercuantil está entre 30 y 38 meses (aprox. 2 a 3 años), lo cual está acorde con la media y mediana, es decir, hablamos de una duración de suscripción estable en el conjunto e inclinado a la derecha con respecto al valor mínimo (cero).

Age

Variable categórica ya que en el histograma se ve como un gráfico de barras de frecuencia en lugar de una distribución.

Seconds of Use

La cola está inclinada a la izquierda cuando vemos la distribución de uso en horas y adicional a este punto, al graficar la distribución en minutos de llamadas con duración de 1 hora vemos una distribución prácticamente uniforme, sin embargo, los clientes con un uso elevado incrementa la cantidad de outliers.

Frequency of SMS

La mitad de los clientes manda menos de 21 SMS, sin embargo, vemos que se comienza a disparar los valores a 87 en el tercer cuantil y hasta un valor máximo de 522 SMS, lo cual nos habla de una variable con al menos una cuarta parte compuesta de valores atípicos.

Frequency of Use

- Una cuarta parte de los clientes realiza menos de 21 llamadas
 - El rango intercuantil se encuentra entre 21 y 95 llamadas; y
 - El resto de clientes eleva el promedio de la variable, teniendo un valor máximo de 255 llamadas realizadas.
-

Distinct Called Numbers

El 95% de los clientes repite al menos una llamada al mismo número. Mientras que la frecuencia de uso se encuentra entre 21 y 95 llamadas, la cantidad de números distintos que marca el cliente se encuentra entre 10 y 34 lo cual confirma el enunciado, se repiten llamadas a un mismo número.

Customer Value

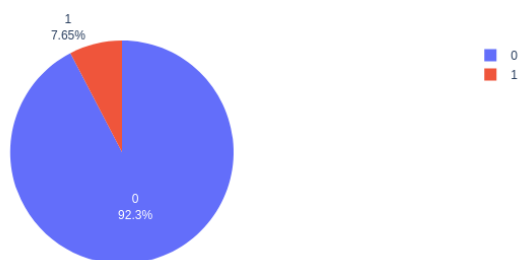
El costo de las llamadas que realizan los clientes se encuentra entre 113 y 788, sin embargo, la mediana está en 228 lo que nos habla de valores elevados ya que la media se encuentra en 500.

De los gráficos observados, los histogramas y gráficos de caja podríamos inferir que en efecto hay valores atípicos ya que en más de una variable observamos alta concentración en un intervalo de valores, sin embargo, necesitamos entender la naturaleza de nuestros datos para determinar si son atípicos o no.

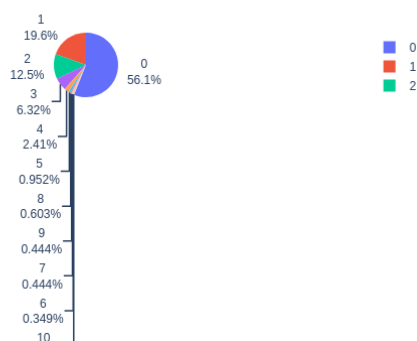
Analiza las variables categóricas. Considera normalizarlas si lo crees necesario.

```
[19]: analisis.plot_cat()
```

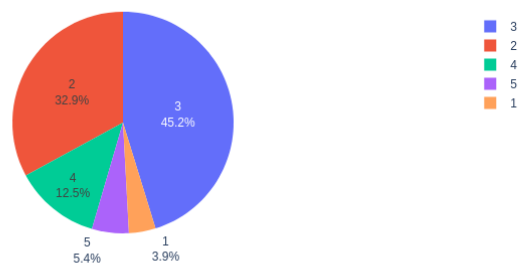
Variable: Complains



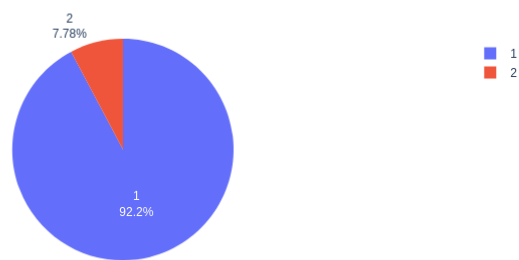
Variable: Charge Amount



Variable: Age Group



Variable: Tariff Plan



Variable: Status



Variable: Churn



Complains

Alta concentración en el valor cero por lo que puede no aportar al modelo incluirla.

Charge Amount

Alta cantidad de categorías, quizás convenga recategorizar esta variable.

Age Group

Alta concentración en los valores 2 y 3 por lo que quizás convenga recategorizar esta variable. Además, observamos que cada categoría representa un número de edad, no un intervalo de edades por lo que podríamos pensar que ya es una columna categórica de la variable edad.

Tariff Plan

Alta concentración en el valor 1 (prepago) por lo que puede no aportar al modelo.

Status

Se encuentra con mejor distribución de ambos valores encontrados 1 (activo) y 2 (inactivo).

```
[20]: # Clasificación de la variable objetivo por si el cliente marcó para una queja
      ↪ 0 no
      aux = df.loc[:, ['Churn', 'Complains', 'Age']]
      aux['Churn_recat'] = aux['Churn'].apply(lambda x: 'Abandono' if x == 1 else
      ↪ 'Retención')
      aux['Complains_recat'] = aux['Complains'].apply(lambda x: 'Queja' if x == 1
      ↪ else 'Sin Queja')
      aux.groupby(['Churn_recat', 'Complains_recat']).agg('count').iloc[:, 0].
      ↪ reset_index(name='Count')
```



```
[20]: Churn_recat  Complains_recat  Count
0      Abandono      Queja      200
1      Abandono      Sin Queja    295
2      Retención     Queja      41
3      Retención     Sin Queja   2614
```

Sí, vamos a normalizar Age Group y Charge Amount después de recategorizar la variable, así como Status y Tariff Plan.

Remueve valores atípicos si lo consideras pertinente. Justifica el porqué.

Por los histogramas y gráficos de caja podríamos pensar que hay outliers debido a que en algunas variables tenemos alta concentración de valores en un rango y el resto puedan estar muy por encima o por debajo de ese rango intercuantil, sin embargo, por la naturaleza del dataset no observamos algo fuera de lo común como que un cliente haya realizado menos una llamada o haya tenido menos una llamada fallida, por lo que consideramos que el dataset no tiene valores atípicos; quizás, sí podemos decir que algunas variables no guardan una distribución uniforme. Además cada cliente tiene un grado distinto de necesidad de realizar una llamada y eso hace que tengamos estos outliers.

Explora correlaciones entre las variables numéricas. ¿Tienen sentido estas correlaciones? ¿Existen problemas?

```
[21]: analisis.corr_matrix('magma', values=True)
```

*Si observamos una correlación positiva entre algunas de las variables y hacen perfecto sentido * Tiempo de uso (en segundos) y frecuencia de uso (0.9) * Costo del cliente y la cantidad de SMS enviados (0.9) * Cantidad de números distintos marcados y la frecuencia de uso (0.7) * Cantidad de llamadas fallidas y frecuencia de uso (0.6) * Costo del clientes y la frecuencia de uso (0.4)*

Sí pudiera haber problemas de multicolinealidad ya que vemos la mitad de las variables correlacionadas positivamente, las que menos se relacionan con las otras variables son: duración de suscripción, costo del cliente, frecuencia de sms enviados y la edad.

Identifica variables que parecen influir más en Churn.

```
[22]: def stats_churn(churn: int): print(f'Churn = {churn}') ; display(df.
      ↪query(f'Churn == {churn}').loc[:, var_num].describe(percentiles=perc).T)

stats_churn(0)
stats_churn(1)
```

Churn = 0

	count	mean	std	min	1% \
Distinct Called Numbers	2655.0	25.582674	17.389349	0.0	0.0
Customer Value	2655.0	535.511501	536.214629	0.0	0.0
Seconds of Use	2655.0	5014.224105	4312.742630	0.0	0.0
Frequency of use	2655.0	76.979284	58.499847	0.0	0.0
Subscription Length	2655.0	32.662524	8.392357	3.0	8.0
Frequency of SMS	2655.0	83.871563	118.808594	0.0	0.0
Age	2655.0	31.065913	9.147610	15.0	15.0
Call Failure	2655.0	7.656121	7.154318	0.0	0.0

	5%	25%	50%	75%	95%	\
Distinct Called Numbers	2.0000	12.000	23.00	36.0000	58.0000	
Customer Value	26.9265	142.065	268.07	864.5475	1662.8745	
Seconds of Use	395.0000	1819.000	3530.00	6892.5000	15221.5000	
Frequency of use	6.0000	32.000	63.00	104.0000	192.3000	
Subscription Length	15.0000	29.000	35.00	38.0000	42.0000	
Frequency of SMS	0.0000	7.000	25.00	133.0000	367.0000	
Age	25.0000	25.000	30.00	30.0000	55.0000	
Call Failure	0.0000	1.000	6.00	12.0000	22.0000	

	99%	max
Distinct Called Numbers	81.460	97.00
Customer Value	2071.575	2165.28
Seconds of Use	16192.300	17090.00
Frequency of use	241.000	255.00
Subscription Length	45.000	47.00
Frequency of SMS	485.000	522.00
Age	55.000	55.00
Call Failure	30.000	36.00

Churn = 1

	count	mean	std	min	1%	5%	\
Distinct Called Numbers	495.0	12.391919	10.867622	0.0	0.0	0.0	
Customer Value	495.0	124.811414	129.429850	0.0	0.0	0.0	
Seconds of Use	495.0	1566.632323	1539.203365	0.0	0.0	0.0	
Frequency of use	495.0	29.133333	26.323478	0.0	0.0	0.0	
Subscription Length	495.0	31.894949	9.469163	3.0	4.0	8.0	
Frequency of SMS	495.0	15.802020	23.515289	0.0	0.0	0.0	
Age	495.0	30.636364	6.886081	25.0	25.0	25.0	
Call Failure	495.0	7.476768	7.831407	0.0	0.0	0.0	

	25%	50%	75%	95%	99%	\
Distinct Called Numbers	2.000	10.00	20.0000	33.000	37.0600	
Customer Value	38.375	96.84	181.3225	335.235	509.4366	
Seconds of Use	318.000	1182.00	2391.5000	5513.000	5865.1000	
Frequency of use	6.000	25.00	45.5000	91.300	98.0000	
Subscription Length	31.000	35.00	37.0000	41.000	43.0600	
Frequency of SMS	0.000	9.00	23.0000	48.000	79.7800	
Age	25.000	30.00	30.0000	45.000	45.0000	
Call Failure	0.000	5.00	11.0000	24.300	30.0000	

	max
Distinct Called Numbers	48.000
Customer Value	987.255
Seconds of Use	6123.000
Frequency of use	100.000
Subscription Length	45.000
Frequency of SMS	204.000

```
Age          55.000
Call Failure 34.000
```

```
[23]: pal = {0: "#FBCEB1", 1: "#7570b3"} # paleta de colores para la variable
      ↪ objetivo

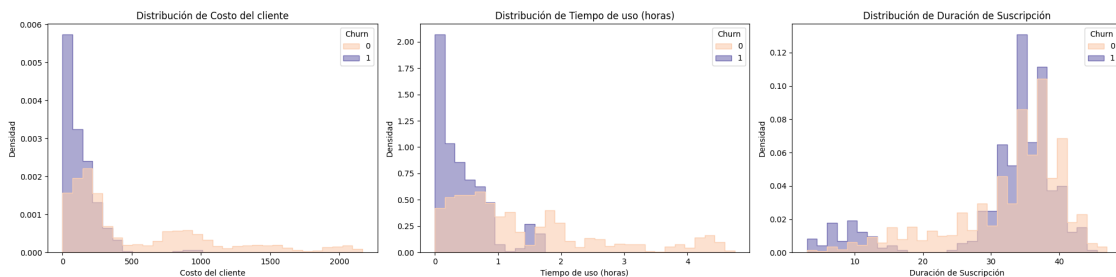
def dist_plot(ax, serie: pd.Series, xlabel: str) -> None:
    global pal

    sns.histplot(
        data=df,
        x=serie,
        hue="Churn",
        hue_order=[0, 1],
        palette=pal,
        bins=30,
        element='step',
        stat='density',
        multiple="layer",
        common_norm=False,
        alpha=0.6,
        ax=ax
    )
    ax.set_title(f'Distribución de {xlabel}')
    ax.set_xlabel(xlabel)
    ax.set_ylabel('Densidad')

fig, axes = plt.subplots(1, 3, figsize=(20, 5))

dist_plot(axes[0], df['Customer Value'], 'Costo del cliente')
dist_plot(axes[1], df["Seconds of Use"]/3600, 'Tiempo de uso (horas)')
dist_plot(axes[2], df["Subscription Length"], 'Duración de Suscripción')

plt.tight_layout(); plt.show()
```



Vemos una posible relación entre el tiempo de uso y la variable objetivo, así como el costo del cliente.

7 Codificación de variables categóricas

```
[24]: var_nom = ['Complains', 'Tariff Plan', 'Status']
      var_ord = ['Age Group', 'Charge Amount']
```

Variables ordinales

*Decidimos normalizar la variable **Charge Amount** dado que se vemos que la categorías de 7 a 10 están demasiado pulverizadas por lo cual procedemos a clasificarla de [0,2] en bajo, [3,5] en medio y [6,10] en alto.*

```
[25]: labels = ['bajo', 'medio', 'alto']

df['Charge Amount_recat'] = pd.Categorical(
    pd.cut(df["Charge Amount"], bins=[0,3,5,10], labels=labels,
    ↪include_lowest=True),
    categories=labels,
    ordered=True
)

display(df['Charge Amount_recat'].tail(3))
```

```
3147    medio
3148     bajo
3149     bajo
Name: Charge Amount_recat, dtype: category
Categories (3, object): ['bajo' < 'medio' < 'alto']
```

Categorizamos la variable Charge Amount.

```
[26]: df['OE_Charge Amount'] = OrdinalEncoder(categories=[labels]).fit_transform(df.
    ↪loc[:, ['Charge Amount_recat']])
```

```
[27]: df.groupby(['OE_Charge Amount', 'Charge Amount_recat'], observed=True).
    ↪agg('count').iloc[:, 1].reset_index(name='Count')
```

```
[27]:  OE_Charge Amount Charge Amount_recat Count
0          0.0          bajo      2979
1          1.0          medio       106
2          2.0          alto        65
```

A continuación el ejemplo de que si importa poner las categorías para que se respete el orden de la variable categórica.

```
[28]: df['OE2_Charge Amount'] = OrdinalEncoder().fit_transform(df.loc[:, ['Charge A
    ↪mount_recat']])
```

```
[29]: df.groupby(['OE2_Charge Amount', 'Charge Amount_recat'], observed=True).
    ↪agg('count').iloc[:, 1].reset_index(name='Count')
```

```
[29]:
```

	OE2_Charge	Amount	Charge	Amount_recat	Count
0		0.0		alto	65
1		1.0		bajo	2979
2		2.0		medio	106

Categorizamos la variable Age Group.

```
[30]: df['OE_Age Group'] = OrdinalEncoder(categories=[[1,2,3,4,5]]).fit_transform(df.
↳loc[:,['Age Group']])
```

```
[31]: df.groupby(['OE_Age Group', 'Age Group'], observed=True).agg('count').iloc[:,1]
↳.reset_index(name='Count')
```

```
[31]:
```

	OE_Age Group	Age Group	Count
0	0.0	1	123
1	1.0	2	1037
2	2.0	3	1425
3	3.0	4	395
4	4.0	5	170

Variables nominales

```
[32]: for x in var_nom:
print(f'Valores únicos en variable {x}:', df[x].unique().__len__())
```

Valores únicos en variable Complains: 2

Valores únicos en variable Tariff Plan: 2

Valores únicos en variable Status: 2

Como hay baja cardinalidad vamos aplicar one-hot encoder.

```
[33]: for x in var_nom:
df[f'OHE_{x}'] = OneHotEncoder(drop='first', sparse_output=False).
↳fit_transform(df[[x]])
display(df.loc[:, [x, f'OHE_{x}', 'Age']].groupby([x, f'OHE_{x}']).
↳agg('count').iloc[:, 0].reset_index(name='Count'))
```

	Complains	OHE_Complains	Count
0	0	0.0	2909
1	1	1.0	241

	Tariff Plan	OHE_Tariff Plan	Count
0	1	0.0	2905
1	2	1.0	245

	Status	OHE_Status	Count
0	1	0.0	2368
1	2	1.0	782

8 Escalamiento de variables numéricas

```
[40]: def plot_scaled_dist(prefix: str, encoder):
    global var_num

    new_columns = [f'{prefix}_{x}' for x in var_num]
    df.loc[:, new_columns] = encoder.fit_transform(df.loc[:, var_num])
    analisis2 = analyze(df.loc[:, new_columns], [f'{prefix}_{x}' for x in
↪var_num], [])
    analisis2.plot_num()

[55]: #plot_scaled_dist('MinMax', MinMaxScaler())

[95]: #plot_scaled_dist('Standard', StandardScaler())

[54]: #plot_scaled_dist('Robust', RobustScaler())

[191]: class Scalers:
    # paletas por método (0 = churn=0, 1 = churn=1)
    pals = {
        "MinMax": {0: "#4F81BD", 1: "#aec7e8"}, # azul medio / azul pastel
        "Standard": {0: "#C0504D", 1: "#ff9896"}, # rojo ladrillo / rojo pastel
        "Robust": {0: "#9BBB59", 1: "#98df8a"} # verde oliva / verde claro
    }

    def __init__(self, df: pd.DataFrame, nvars: list[str]):
        self.X = df.copy()
        self.nvars = nvars
        if "Churn" in self.X.columns:
            self.X["Churn"] = self.X["Churn"].astype(int)

    def __call__(self, alpha = 0.6):#, transformation: str / None = None):
        n = len(self.nvars)
        fig, axes = plt.subplots(n*2, 3, figsize=(20, 4*2))
        axes = np.atleast_2d(axes) # asegura forma (n, 3)

        r = 0
        for var in self.nvars:
            # crea columna log sólo si no existe
            log_col = f'Log {var}'
            if log_col not in self.X.columns:
                self.X.loc[:, log_col] = np.log1p(self.X.loc[:, var])

            self._plot_scaled_var('', var, r, axes, alpha)
            self._plot_scaled_var('Log ', var, r + 1, axes, alpha)
            r += 1
```

```

plt.tight_layout()
plt.show()

def _plot_scaled_var(self, prefix: str, var: str, row: int, axes, alpha):
    base_col = prefix + var

    aux = self.X.loc[:, [base_col]].copy()
    aux['MinMax'] = MinMaxScaler().fit_transform(aux[[base_col]])
    aux['Standard'] = StandardScaler().fit_transform(aux[[base_col]])
    aux['Robust'] = RobustScaler().fit_transform(aux[[base_col]])
    aux['Churn'] = self.X['Churn'].values

    # MinMax
    self._plot_by_churn(aux, "MinMax", axes[row, 0], "MinMax", base_col,
↪alpha)
    # Standard
    self._plot_by_churn(aux, "Standard", axes[row, 1], "Standard",
↪base_col, alpha)
    # Robust
    self._plot_by_churn(aux, "Robust", axes[row, 2], "Robust", base_col,
↪alpha)

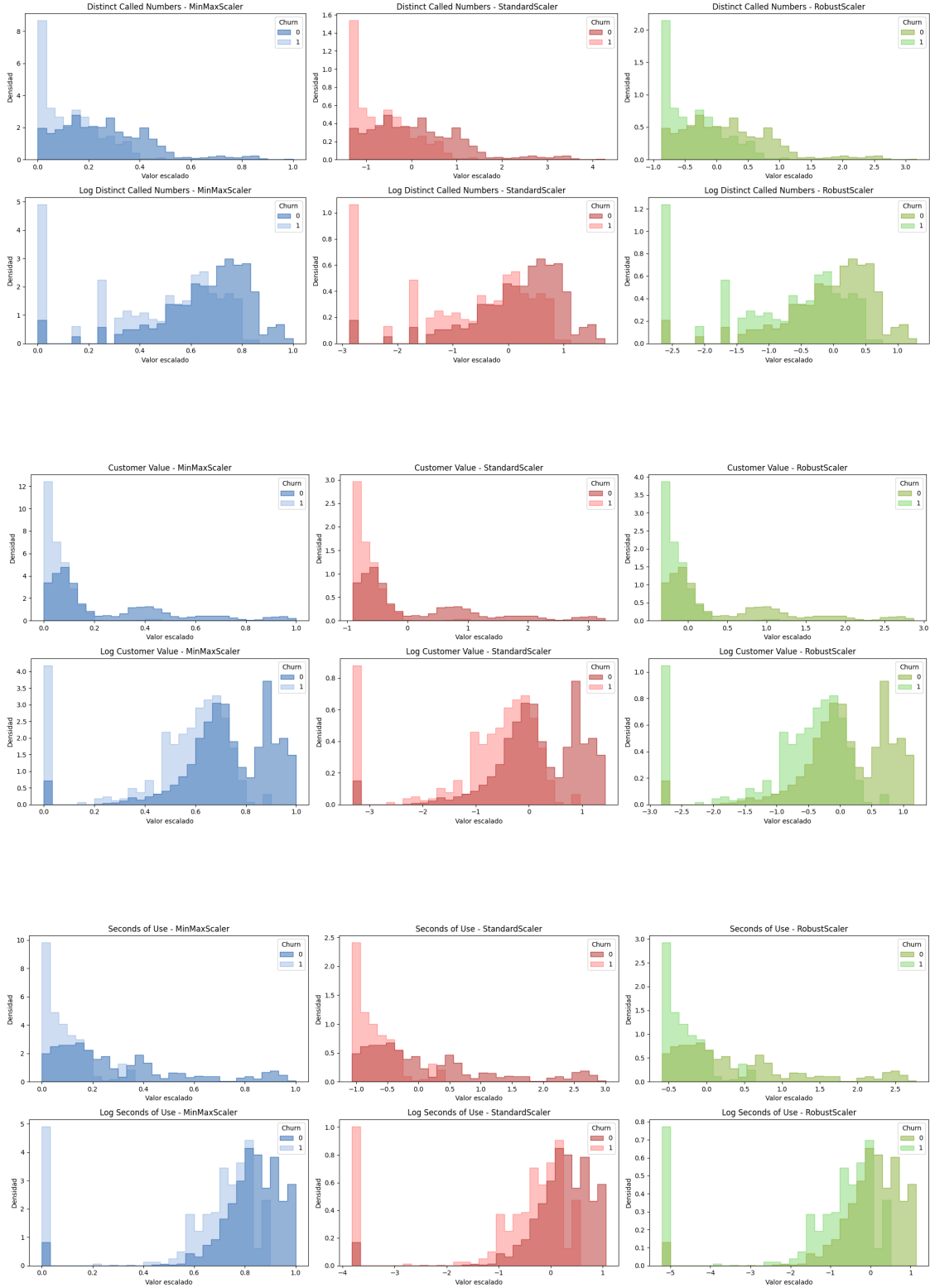
def _plot_by_churn(self, data, col, ax, method, base_col, alpha=0.35):
    sns.histplot(
        data=data,
        x=col,
        hue="Churn",
        stat="density",
        element="step",
        multiple="layer",
        common_norm=False,
        bins=30,
        alpha=alpha,
        palette=self.pals[method], # usa paleta por método
        ax=ax
    )
    ax.set_title(f"{base_col} - {method}Scaler")
    ax.set_xlabel("Valor escalado")
    ax.set_ylabel("Densidad")

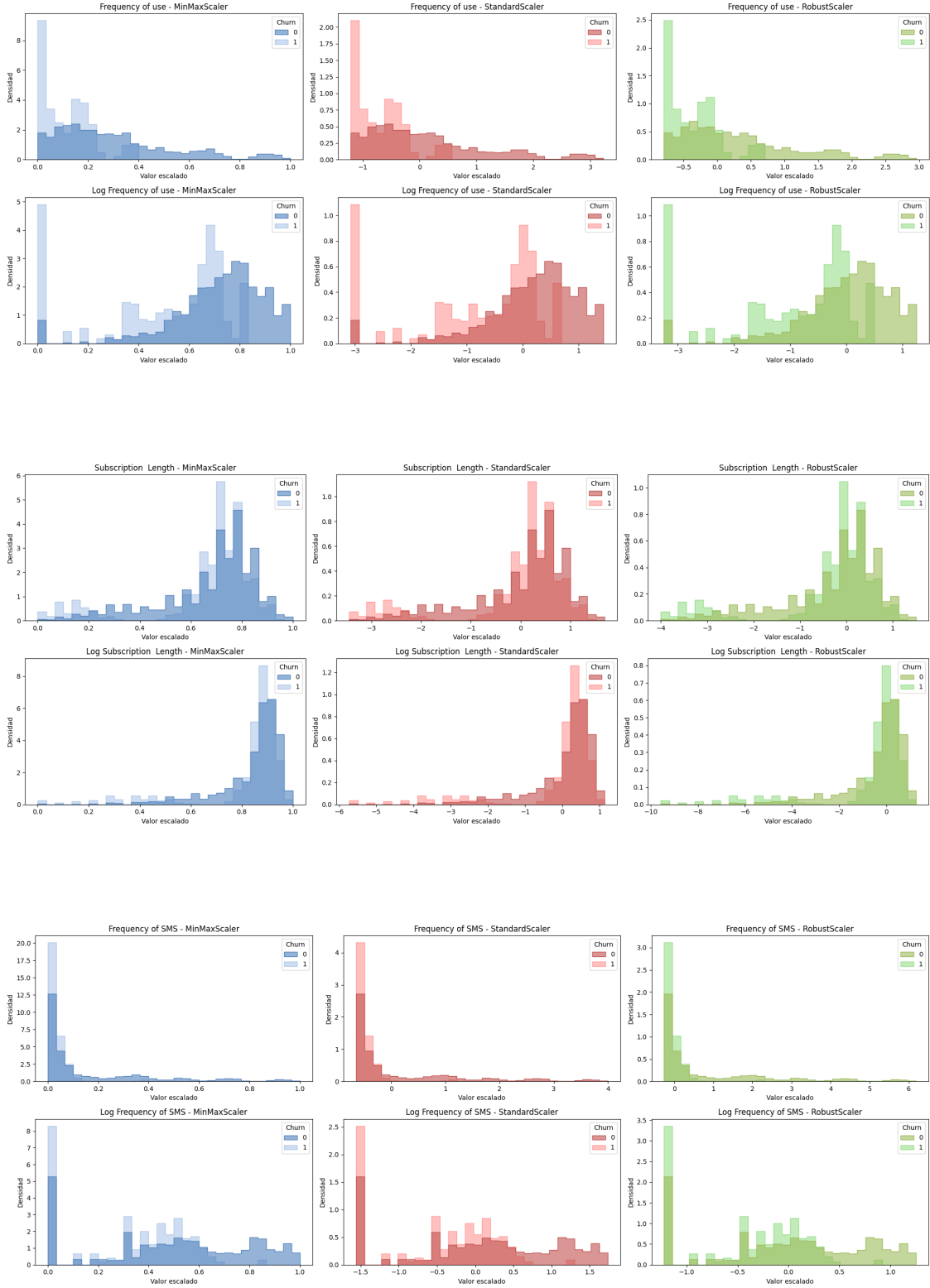
```

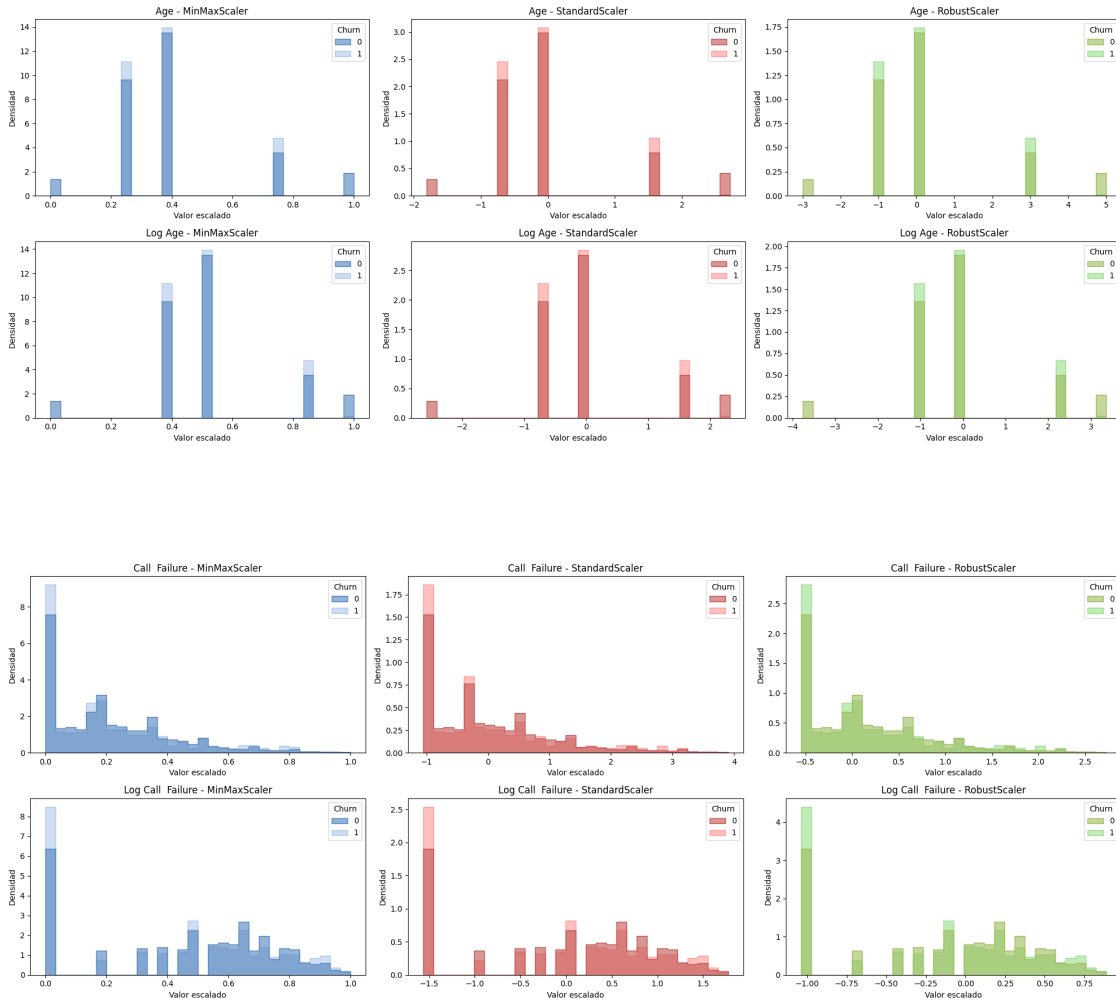
```

[196]: for x in var_num:
        Scalers(df, [x])()

```







Después de observar los resultados anteriores vemos conveniente aplicar log sobre las variables numéricas para reducir el efecto de las colas pesadas y así tener una mejor concentración de los valores para el modelo.

```
[204]: new_var_cat = [x for x in df.columns if x.startswith('OHE') or x.
    ↪startswith('OE')]
```

```
[205]: new_var_cat.pop(1)
```

```
[205]: 'OE2_Charge Amount'
```

```
[206]: print(new_var_cat)
```

```
['OE_Charge Amount', 'OE_Age Group', 'OHE_Complains', 'OHE_Tariff Plan',
'OHE_Status']
```

```
[221]: new_var_num = set(var_num)-set(['Age'])
```

```
[222]: for x in new_var_num:
        df.loc[:, f'MinMax_{x}'] = np.log1p(df.loc[:, x])
```

```
[223]: new_var_num = [f'MinMax_{x}' for x in new_var_num]
```

```
[224]: X_Scaled = pd.concat([
        df.loc[:, new_var_cat],
        pd.DataFrame(data=MinMaxScaler().fit_transform(df.loc[:, new_var_num]),
        ↪columns=new_var_num)],
        axis=1
    )
```

```
[225]: X_Scaled
```

```
[225]:      OE_Charge  Amount  OE_Age Group  OHE_Complains  OHE_Tariff Plan \
0              0.0        2.0          0.0          0.0          0.0
1              0.0        1.0          0.0          0.0          0.0
2              0.0        2.0          0.0          0.0          0.0
3              0.0        0.0          0.0          0.0          0.0
4              0.0        0.0          0.0          0.0          0.0
...          ...          ...          ...          ...          ...
3145           0.0        1.0          0.0          0.0          1.0
3146           0.0        4.0          0.0          0.0          0.0
3147           1.0        2.0          0.0          0.0          0.0
3148           0.0        2.0          0.0          0.0          0.0
3149           0.0        2.0          1.0          0.0          0.0
```

```
      OHE_Status  MinMax_Distinct Called Numbers  MinMax_Customer Value \
0              0.0              0.630402          0.688928
1              1.0              0.351025          0.501368
2              0.0              0.702050          0.955364
3              0.0              0.781580          0.714106
4              0.0              0.769114          0.649558
...          ...          ...          ...          ...
3145           0.0              0.830249          0.857126
3146           0.0              0.820333          0.725077
3147           0.0              0.674169          0.734236
3148           0.0              0.559426          0.909213
3149           0.0              0.502203          0.601741
```

```
      MinMax_Seconds of Use  MinMax_Frequency of use \
0              0.860095          0.771241
1              0.591526          0.323120
2              0.800865          0.741342
3              0.855976          0.758261
4              0.798325          0.735330
...          ...          ...
```

3145	0.903887	0.901182
3146	0.936876	0.934467
3147	0.826743	0.712555
3148	0.867453	0.694324
3149	0.768665	0.587555

	MinMax_Subscription	Length	MinMax_Frequency of SMS \
0		0.916440	0.286243
1		0.926628	0.332201
2		0.905986	0.940335
3		0.916440	0.110734
4		0.916440	0.175509
...		...	...
3145		0.647685	0.724106
3146		0.605285	0.702036
3147		0.627044	0.585273
3148		0.442114	0.863823
3149		0.442114	0.332201

	MinMax_Call	Failure
0		0.608495
1		0.000000
2		0.664068
3		0.664068
4		0.383917
...		...
3145		0.856027
3146		0.800453
3147		0.730855
3148		0.575876
3149		0.608495

[3150 rows x 12 columns]

9 Variables Explicativas

```
[231]: X = X_Scaled.copy()
y = df.loc[:, 'Churn']
```

```
[252]: def is_discrete(s: pd.Series, max_unique=20):
        return s.dtype.kind in "biu" and s.nunique() <= max_unique
```

```
[240]: def woe_iv(df, feature, target, bins=5, discretize=False):
        eps = 0.00001 # para evitar división por cero

        # Discretizar si es numérica
```

```

if discretize:
    df = df.copy()
    df[feature] = pd.qcut(df[feature], q=bins, duplicates="drop")

tmp = pd.crosstab(df[feature], df[target], normalize='columns')

# % buenos y % malos por categoría
tmp.columns = ["%malos", "%buenos"]
tmp["WoE"] = np.log((tmp["%buenos"]+eps) / (tmp["%malos"]+eps))
tmp["IV_contrib"] = (tmp["%buenos"] - tmp["%malos"]) * tmp["WoE"]

iv = tmp["IV_contrib"].sum()
return tmp, iv

```

```

[246]: def IV_df(df, features, target, bins=5, discretize=False):
    results = []
    for feature in features:
        try:
            _, iv = woe_iv(df, feature, target, bins=bins,
↪discretize=discretize)
            results.append({"Variable": feature, "IV": iv})
        except Exception as e:
            print(f"Error con {feature}: {e}")

    results_df = pd.DataFrame(results).sort_values(by="IV", ascending=False).
↪reset_index(drop=True)
    return results_df

```

```

[253]: discrete_mask = np.array([is_discrete(X[c]) for c in X.columns], dtype=bool)

mi = mutual_info_classif(X.values, y, discrete_features=discrete_mask,
↪random_state=42)
mi_df = pd.DataFrame({'feature': X.columns, 'MI': mi}).sort_values('MI',
↪ascending=False).reset_index(drop=True)
display(mi_df)

```

	feature	MI
0	MinMax_Seconds of Use	0.131483
1	OHE_Status	0.111700
2	MinMax_Customer Value	0.105648
3	OHE_Complains	0.097617
4	MinMax_Frequency of use	0.097133
5	MinMax_Distinct Called Numbers	0.059013
6	MinMax_Frequency of SMS	0.055888
7	MinMax_Subscription Length	0.033822
8	OE_Charge Amount	0.014775
9	OE_Age Group	0.012814

```

10             OHE_Tariff Plan    0.005270
11             MinMax_Call Failure 0.000000

```

```
[248]: IV_df(pd.concat([X[new_var_num], y], axis=1), new_var_num, "Churn", 5, True)
```

```

[248]:
           Variable      IV
0      MinMax_Seconds of Use  2.966552
1      MinMax_Frequency of use  2.900963
2      MinMax_Customer Value  1.445253
3  MinMax_Distinct Called Numbers  1.015768
4      MinMax_Frequency of SMS  0.899936
5      MinMax_Subscription Length  0.117483
6      MinMax_Call Failure  0.018096

```

Los resultados muestran que la variable con mayor poder predictivo frente a churn es **Seconds of Use**, seguida de **Frequency of Use** y **Customer Value**, con valores de IV superiores a 1, lo que indica una relación extremadamente fuerte con la variable objetivo. Sin embargo, estos valores tan altos sugieren que podría existir **riesgo de fuga de información (data leakage)** o que las variables capturan de forma directa el comportamiento asociado al churn. Variables como **Subscription Length** presentan un IV moderado (0.11), lo que las hace útiles como predictoras secundarias. En cambio, **Call Failure** (IV = 0.018) no aporta información relevante y podría ser descartada. En términos de **Mutual Information**, los patrones son consistentes: Seconds of Use, Customer Value y Status destacan con mayor dependencia respecto al churn, confirmando su relevancia.

Nota. El data leakage o riesgo de fuga de información se refiere a variables con alto valor IV el cual nos habla de una muy buena relación con la variable objetivo, lo cual buscamos, sin embargo, si nos referimos a información que no se encuentra disponible en ese momento quiere decir que es información pérdida o fugada ya que no puede entrar al modelo si no se encuentra disponible en ese momento, sin embargo, si se realizan distintos cortes podemos conocer esa información (proporcional al cierre de mes) para incluirla al modelo y mejorar la predicción del mismo.

10 Aplicación de PCA

En este caso **no es necesario aplicar PCA**, ya que las transformaciones realizadas (log y Min-MaxScaler) fueron suficientes para estabilizar las variables y llevarlas a una misma escala. Además, las distribuciones no presentan un comportamiento cercano a gaussiano, por lo que la reducción con PCA no aportaría ventajas significativas. Dado que el número de variables es manejable, es preferible conservarlas en su forma transformada para no perder interpretabilidad.

```

[254]: class rPCA:
        #Inicializamos
        def __init__(self, dataframe, cols, n_comp):
            self.dataframe = dataframe
            self.cols = cols
            self.n_comp = n_comp

            if self.cols == None:

```

```

        self.dataframe = dataframe
    else:
        self.dataframe = dataframe[cols]

#Ajustamos el PCA
    def r_pca(self):
        pca = PCA(n_components = self.n_comp)
        pca.fit(self.dataframe)
        self.df_trans = pd.DataFrame(pca.transform(self.dataframe),
        ↪columns=[f'p_{i}' for i in range(len(pca.explained_variance_ratio_))])
        return self.df_trans, pca.explained_variance_ratio_.cumsum()

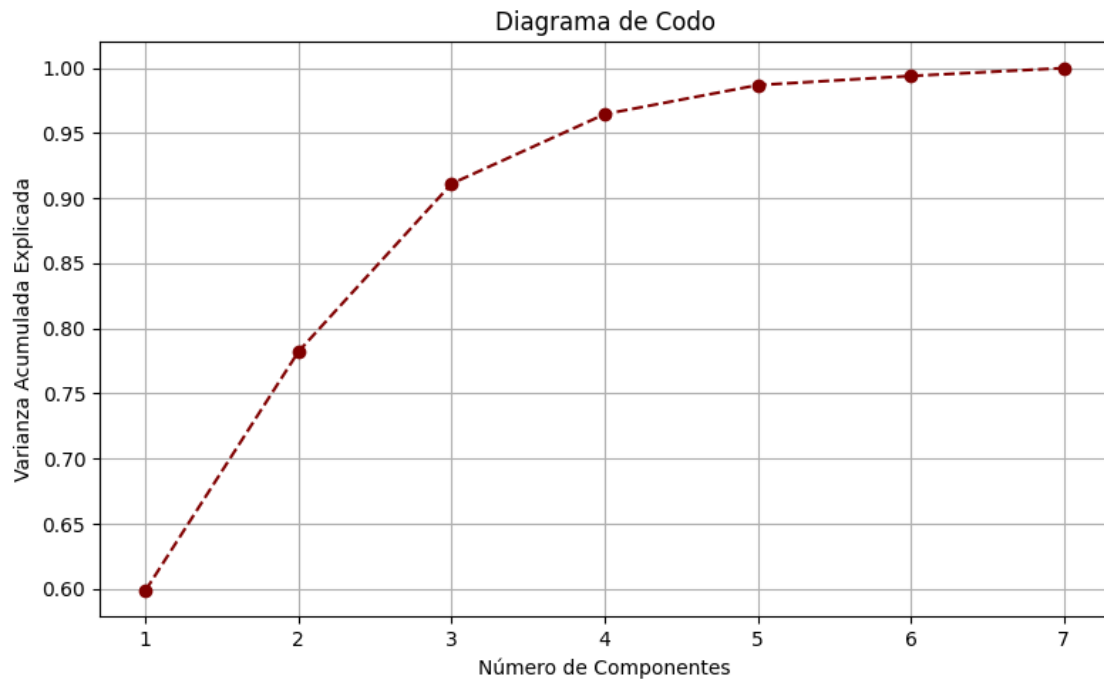
#Diagrama del codo
    def plot_elbow(self):
        pca_temp = PCA().fit(self.dataframe)
        var_exp = pca_temp.explained_variance_ratio_
        cum_var_exp = var_exp.cumsum()

        plt.figure(figsize=(8, 5))
        plt.plot(range(1, len(var_exp) + 1), cum_var_exp, marker='o',
        ↪linestyle='--', color='maroon')
        plt.xlabel('Número de Componentes')
        plt.ylabel('Varianza Acumulada Explicada')
        plt.title('Diagrama de Codo')
        plt.grid(True)
        plt.tight_layout()
        plt.show()

```

```
[255]: pca_01 = rPCA(X, new_var_num, 0.90)
```

```
[256]: pca_01.plot_elbow()
```



```
[257]: print("Componentes:", pca_01.r_pca()[0].shape[1])
```

Componentes: 3

Esto confirma lo observado en la tabla de IV y mutual information, tenemos alta predicción con pocas variables y al estar trabajando con una cantidad manejable de variables no vemos necesarios aplicar PCA.